The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

6

Products, Categories and Their Tables

Part II — Standing the Shop Up

Taught in	Week 4
Laboratory	50,000 products, and the index that rescues the category page
Assessment	—

This is a class extract. It is *Chapter 6* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 6

Products, Categories and Their Tables

Part 1 | **Part 2** | Part 3 | Part 4 | Part 5 | Part 6

Part II — Standing the Shop Up

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why one product occupies eighteen tables, and **say** what decides which table a given fact belongs in.
- **Create** products and categories through the admin panel, and **find** in the database exactly what each screen wrote.
- **Describe** how a category hierarchy is stored, and **say** what the path table buys and what it costs.
- **Read** the query a category page runs, and **use** `EXPLAIN` to say what the database does with it.
- **Measure** how the catalogue's size affects the page, and **decide**, from evidence, whether an index would help.

Before You Start

Chapter 5: a running shop. *Database Systems* for joins, keys and indexes; Appendix B if those need refreshing. You will spend as much of this chapter in the database console as in the admin panel.

Key Terms in This Chapter

catalogue • product • category • SKU • model • normalisation • junction table • many-to-many • localisation • materialised path • denormalisation • index • full table scan • filesort • correlated subquery • pagination

6.1 One Product Is Not One Row

Ask a student to design a products table and you get something like this:

```
CREATE TABLE product (
  product_id  INT PRIMARY KEY,
  name        VARCHAR(255),
  description TEXT,
  price       DECIMAL(10,2),
  image       VARCHAR(255),
  category    VARCHAR(100)
);
```

It is a reasonable first attempt and every part of it is wrong in a way that matters. Your shop has **eighteen** tables whose names begin `oc_product`, and the reasons for each are worth knowing, because the same reasons will apply to whatever you build after this course.

```
SELECT table_name FROM information_schema.tables
 WHERE table_schema = 'opencart'
   AND table_name LIKE 'oc_product%'
 ORDER BY table_name;
```

Four questions that split a table

Every one of those eighteen tables exists because of one of four questions. Learn the questions and the table list stops being arbitrary.

1. **"Does this fact depend on the language?"** A price does not; a name does. So `price` lives in `oc_product` and `name` lives in `oc_product_description`, keyed on *both* the product and the language.
2. **"Can there be more than one?"** A product has one price and many photographs, so photographs are rows in `oc_product_image` rather than columns.
3. **"Does this relate two things that both exist already?"** A product belongs to several categories and a category holds several products. Neither can hold the other, so a junction table — `oc_product_to_category` — holds the relationship.
4. **"Is this about the product, or about something that happened to it?"** `oc_product_viewed` and `oc_product_report` record events, not properties, and grow without bound.

6.1.1 The two tables you will use constantly

```
DESCRIBE oc_product;              -- the language-independent facts
DESCRIBE oc_product_description;  -- the words, per language
```

`oc_product` holds what is true regardless of who is reading: `price`, `quantity`, `weight`, `status`, `date_available`, `sort_order`. Its primary key is `product_id`.

`oc_product_description` holds `name`, `description`, `meta_title` and the rest — and its primary key is the *pair* `(product_id, language_id)`. One product, one row per language.

Why the split is not over-engineering

It looks like extra work for a shop that will only ever be in English. It is not, and the reason is worth more than the example.

Suppose name and price shared a row, and the shop later added Urdu. You now need a second row for the Urdu name — which duplicates the price. Change the price and you must remember to change it in both rows, and the day somebody forgets, the shop charges two different prices depending on the language the customer is reading. That is not a bug that gets noticed quickly; it is a bug that gets noticed by an accountant, months later.

The split makes that mistake *impossible to express*. There is exactly one place the price lives, so there is exactly one price. This is what normalisation is for, and it is the clearest small example of it you will meet.

6.2 The Screen, and What It Writes

Create a product: **Catalog › Products › Add New**. The form has tabs — **General**, **Data**, **Links**, **SEO** — and the tabs are not a presentational choice. They are very nearly the table list.

Tab	What you type	Where it lands
General	Product name, description, meta title and tags	One row in `oc_product_description`, for the language you were editing
Data	Model, price, quantity, weight, dimensions, date available, status	The single row in `oc_product`

Tab	What you type	Where it lands
Links	Manufacturer, categories, stores, related products	Rows in `oc_product_to_category`, `oc_product_to_store` and `oc_product_related` — one per tick
Image	Main image and additional images	`image` on the product row; the extras become rows in `oc_product_image`
SEO	The URL keyword	`oc_seo_url` — not a product table at all, which Chapter 21 explains

Do this once, deliberately, and you will never be confused again

Create one product. Then, in the database console at `http://localhost:8091/`, run:

```
SELECT MAX(product_id) FROM oc_product;   -- yours, the newest
```

and look at that id in `oc_product`, then in `oc_product_description`, then in `oc_product_to_category`. Three tables, one product, and you typed it all on one screen.

Every chapter after this one assumes you can do this. When something in the shop is not what you expected, the question is always "what did it actually write?", and the admin panel is the wrong place to look for the answer.

6.3 Categories, and the Table That Is Not Normalised

Categories form a tree: **Components › Monitors** sits under **Components**, which sits at the top. The obvious storage is a `parent_id` on each category, and that is indeed what `oc_category` has.

It is not enough, and the reason is the one interesting thing about category storage.

The question a parent pointer cannot answer cheaply

A customer opens **Components**. The page must show every product in *Components or anything beneath it*, however deep.

With only `parent_id`, answering that means: find the children of Components; find their children; find theirs; stop when there are no more. That is a loop with a query inside it, and the number of queries depends on how deep the tree happens to be — so the page gets slower as the shop's categories get better organised, which is precisely the wrong incentive.

So OpenCart keeps a second table that stores the answer directly.

```
DESCRIBE oc_category_path;
-- category_id, path_id, level

SELECT * FROM oc_category_path WHERE category_id = 27 ORDER BY level;
-- category_id  path_id  level
--          27       20      0      <- Components, the ancestor
--          27       27      1      <- Monitors, itself
```

Materialised Path

A **materialised path** stores, for every node, the complete list of its ancestors — one row per ancestor, with the depth. The tree is still in `parent_id`; this table is a *precomputed answer* to "who is above me?", and therefore also to "who is below me?"

It is a deliberate **denormalisation**: the same fact is now stored twice, in `oc_category.parent_id` and in `oc_category_path`. That buys a single-query answer to the

descendants question, and costs the obligation to keep the two in step — so moving one category in the admin panel rewrites every path row beneath it.

This is the trade-off in miniature: **denormalisation buys read speed with write complexity and the risk of disagreement.** Make it deliberately, and only where the reads vastly outnumber the writes — which for a category tree they do, by perhaps a million to one.

6.4 What the Category Page Actually Asks

Here is where this chapter stops being about design and starts being about evidence. The shop's own source is in your container, and you can read it:

```
docker compose exec shop \
  cat /var/www/html/catalog/model/catalog/product.php
```

The listing query is built up in `getProducts()`, and reduced to its skeleton it looks like this:

```
SELECT DISTINCT *, pd.name, p.image,
       (SELECT ... FROM oc_product_discount pd2
         WHERE pd2.product_id = p.product_id ... LIMIT 1) AS discount,
       (SELECT ... FROM oc_product_discount ps
         WHERE ps.product_id = p.product_id ... LIMIT 1) AS special,
       (SELECT pr.points FROM oc_product_reward pr
         WHERE pr.product_id = p.product_id ...)         AS reward,
       (SELECT COUNT(*) FROM oc_review r
         WHERE r.product_id = p.product_id ...)          AS reviews
  FROM oc_category_to_store c2s
  LEFT JOIN oc_category_path cp  ON (cp.category_id = c2s.category_id)
  LEFT JOIN oc_product_to_category p2c ON (p2c.category_id = cp.category_id)
 INNER JOIN oc_product_to_store p2s ON (p2s.product_id = p2c.product_id)
  LEFT JOIN oc_product p  ON (p.product_id = p2s.product_id
                              AND p.status = 1
                              AND p.date_available <= NOW())
  LEFT JOIN oc_product_description pd ON (p.product_id = pd.product_id)
 WHERE ...
 ORDER BY p.sort_order
 LIMIT 0, 20;
```

Four things in that query are worth naming, because each is a decision somebody made and each has a consequence you can measure.

1. **`SELECT DISTINCT *`** — every column of every joined table, then deduplicated. The `DISTINCT` is there because the path join can produce a product twice; it forces a temporary table.
2. **Four correlated subqueries.** Discount, special, reward and review count are each evaluated *per row returned*. With `LIMIT 20` that is eighty extra queries' worth of work per page, and it is the SQL form of the N+1 problem.
3. **`ORDER BY p.sort_order`** is the default sort, and `oc_product` has **no index on `sort_order`** — or on `price`, `status` or `date_available`. It has a primary key and nothing else.
4. **The whole join runs twice.** `getTotalProducts()` repeats it with `COUNT(DISTINCT p.product_id)`, because the page needs to know how many pages there are. Every category page you have ever loaded ran that join twice.

</> Reading EXPLAIN on the real query

Put `EXPLAIN` in front of it and the database says what it intends:

```
p:ALL [Using where; Using temporary; Using filesort]
```

Three findings in one line. `ALL` is a full table scan of `oc_product` — every row read, at any catalogue size. `Using temporary` is the `DISTINCT` building a temporary table. `Using filesort` is the `ORDER BY` sorting the result afterwards, because no index offers it in that order.

Now ask for the same page sorted by name, which is `ORDER BY LCASE(pd.name)`:

```
pd:index [Using where; Using index; Using temporary; Using filesort]
```

Still a filesort — and `oc_product_description` *does* have an index on `name`. Chapter 6's key lesson is in that contradiction, and the next section measures it.

A function around a column hides the column

`ORDER BY LCASE(pd.name)` cannot use an index on `name`.

An index on `name` stores the *names*, in order. The query does not ask for names in order — it asks for `LCASE(name)` in order, and the database has no index on that. It cannot assume the two orderings agree, because for some collations they do not. So it reads every row, computes `LCASE` on each, and sorts the results.

The rule, which is worth more than this example: the moment you wrap a column in a function in `WHERE` or `ORDER BY`, any index on that column stops being usable. `WHERE YEAR(date_added) = 2026` and `WHERE UPPER(email) = ?` are the same mistake, and they are extremely common.

The fixes are to store the value you actually search by, to use a case-insensitive collation so `LCASE` is unnecessary, or — in modern MariaDB and MySQL — to build an index on the expression itself.

6.5 What It Costs, and What an Index Buys

The argument so far is that this query ought to be slow and that an index ought to help. Both halves are worth checking, and only one of them survives.

Measured: The Category Page Against Catalogue Size

Produced by `code/m06_catalogue.py` on the machine in Appendix A. Products are seeded into one category; the page is requested seven times at each size and the median taken; then the same page is requested once more with MariaDB's query log recording *every* statement it ran and how long each took.
The whole run was done **twice**, because one of its results was surprising enough that a single observation would not have been worth reporting. Both runs are shown.

Products	Page	Queries	DB time	DB as % of page
100	112 / 111 ms	**144**	60 / 35 ms	54% / 32%
1,000	203 / 101 ms	**144**	60 / 60 ms	30% / 60%
10,000	171 / 302 ms	**144**	114 / 128 ms	67% / 43%
50,000	541 / 476 ms	**146 / 144**	402 / 477 ms	74% / 100%

One page, one hundred and forty-four queries — and that is the most stable number in the whole experiment. It is the same at a hundred products as at fifty thousand, in both runs. Twenty products are displayed, each needing its images, its discount, its special price and its review count, and around forty more queries fetch settings, languages, currencies, layout modules and the category tree. The $N + 1$ problem here is per *displayed* product, which is why growing the catalogue does not grow it.

The catalogue's size does cost, and it lands on the database. Five hundred times the products cost roughly 4× the page time and 8–13× the database time, and the database's share rises from about a third to between 74% and 100%. At fifty thousand products this page is database-bound; at a hundred it is not.

Now the surprise. The indexes help in the middle and hurt at the top.

Products	DB before	DB after	Change (run 1 / run 2)
100	60 / 35 ms	43 / 42 ms	−28% / +19% — noise
1,000	60 / 60 ms	41 / 48 ms	**−31%** / **−20%**
10,000	114 / 128 ms	107 / 106 ms	−6% / **−18%**
50,000	402 / 477 ms	616 / 574 ms	**+53%** / **+20%**

At a thousand and ten thousand products the indexes make the database faster, in both runs. At fifty thousand they make it **slower**, in both runs. The direction reproduces even though the magnitudes do not, which is what makes it worth believing.

Why an index can lose. `EXPLAIN` shows what changed: `p:ALL [... Using filesort]` became `p:index [...]`. Given an index on `sort_order`, the optimiser walks the index in order and avoids the sort — but it must then fetch each row by primary key, which is random access. Without the index it reads the table straight through, which is sequential, and sorts afterwards. The `DISTINCT` means it has to touch nearly every row either way, so the question is only whether those reads are sequential or scattered. At fifty thousand rows, **scattered reads plus no sort lose to sequential reads plus a sort.**

The chapter's conclusion, which is not the one the chapter expected. "Add an index" is not a diagnosis, and removing a filesort is not automatically a win. The index is right for a mid-sized catalogue and wrong for a large one, and nothing in `EXPLAIN` told us that — only the clock did. Meanwhile the page still runs 144 queries at every size, which no index touches at all. That is a structural problem, and Chapter 20 meets it with the only tool that works at that level: caching.

⚠ Three earlier versions of this measurement were wrong

All three are recorded because all three mistakes are ones you will make. The third is the worst kind.

Third, and the one that should frighten you: the products were not there. The seeder wrote `oc_product_to_store` with `store_id = 1`. The default store is **0**. Every storefront query filters on `p2s.store_id = config_store_id`, so fifty thousand products sat in the database, appeared in the admin panel, and were invisible to every customer — and to the measurement, which was timing a category page showing the nineteen demonstration products.

Nothing failed. No error, no warning, no empty page: the page rendered perfectly, returned 200, and meant something else. The numbers it produced were real numbers about a different

question, and they were only caught because a *later* chapter tried to open one of the seeded products and got a 404.

That is the characteristic shape of a measurement bug. It does not crash; it quietly answers a question you did not ask. The defence is to check that your setup did what you think before you trust what it tells you — here, one `curl` for a seeded product's page would have caught it in week one.

First: the harness timed itself. It ran the listing query twenty-five times through `docker compose exec` and divided the wall clock by twenty-five. It reported **707 ms of query time inside a 277 ms page** — impossible, and the impossibility was the only reason it was questioned. What it was really measuring was the cost of spawning a process and opening a connection, which dwarfs the query. Timing a trivial statement first and subtracting it fixed the small sizes.

Second, and worse: it was timing the wrong query. The harness used a hand-written query that *resembled* the one OpenCart runs. At 50,000 products it still reported more database time than the whole page took, because the hand-copied version lacked the real one's restrictions and did far more work. Two queries that look alike are not alike.

The fix was to stop guessing. MariaDB will record every statement it executes, with its duration, if you set `long_query_time=0` and turn the slow query log on. That is what produced the table above — the real queries, timed by the server, including the 146 of them.

Two rules, both expensive to learn any other way. **If a measurement gives you a number that cannot be true, believe the impossibility.** And **never hand-copy the query you mean to measure** — ask the server what it ran.

Common Pitfalls

- **Putting the name in `oc_product`.** It is the one column that depends on the language, and the one students most want to move.
- **Editing `oc_category_path` by hand.** It is derived. Move categories in the admin panel and let it rewrite the paths, or the tree and the paths disagree and the storefront shows products in categories they are not in.
- **Wrapping a column in a function and expecting the index to work.** `LCASE(name)`, `YEAR(date_added)`, `UPPER(email)` — all the same mistake.
- **Adding indexes because a page is slow.** Measure first. This chapter added three, removed a full scan and a filesort, and gained nothing.
- **Forgetting that the join runs twice.** Optimising the listing query and ignoring the `COUNT` halves your improvement at best.
- **Testing a catalogue with nineteen products.** Every interesting behaviour in this chapter is invisible below about a thousand.

Try It Yourself: Fifty thousand products, and where the time goes

Your shop has nineteen products. By the end of this laboratory it will have had fifty thousand, and you will know what that costs.

```
# --- 1. Look at what one product is, before changing anything -------
# Create a product in the admin panel: Catalog > Products > Add New.
# Fill in General (name), Data (model, price) and Links (a category).
# Then find it three times, in three tables.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count the tables one product touches ------------------------
```

```
# 18 of them begin oc_product. Read the list and, for each, say which
# of the four questions in this chapter put it there.

# --- 3. Fill the catalogue ---------------------------------------
# Seeded from a fixed seed, so your classmate's shop matches yours.
python seed/catalogue.py 50000

# --- 4. Load the category page and feel it -----------------------
# http://localhost:8090/index.php?route=product/category&path=20
# Then sort by name, and by price. One of those is slower; predict
# which before you try it.

# --- 5. Ask the database what it is doing ------------------------
# EXPLAIN the listing query. Find ALL, Using temporary, Using filesort.
# Then EXPLAIN it with ORDER BY LCASE(pd.name) and explain why the
# existing index on name does not save you.

# --- 6. Run the measurement --------------------------------------
# Four catalogue sizes, with and without the missing indexes. Watch
# the query COUNT as the catalogue grows -- it is the finding.
python m06_catalogue.py

# --- 7. Disagree with the book -----------------------------------
# The book found 146 queries per page, and that the indexes halved
# the database time without reliably moving the page time. Does
# yours? Then answer the question the book leaves open: of those 146
# queries, how many are for the 20 products shown, and how many would
# still run on an EMPTY category? Find out by requesting a category
# with no products in it, with the query log on.

# --- 8. Put your shop back ---------------------------------------
python seed/catalogue.py --restore
```

What to notice. Step 7 is the real exercise. The measurement in this book is a claim about one machine on one afternoon, and your job is not to reproduce the number but to find out whether the *shape* holds on yours — and, if it does, to say where the time is actually going. A student who can answer that can optimise a shop; one who can only add indexes cannot.

Checkpoint — Test Yourself

1. A shop adds Urdu. Which of these move to `oc_product_description` and which stay in `oc_product`, and why: price, product name, weight, meta description, stock quantity?
2. `oc_category_path` stores facts already derivable from `oc_category.parent_id`. Name what this buys, what it costs, and the one operation that makes the cost visible.
3. A colleague reports that a category page is slow, runs `EXPLAIN`, sees `Using filesort`, adds an index, and the page is no faster — although the database's own time halved. Give two distinct explanations for why the page did not improve, and say what you would measure to tell them apart.

Chapter Summary

- One product occupies eighteen tables. Four questions decide which: does the fact depend on language, can there be more than one, does it relate two existing things, and is it a property or an event?
- `oc_product` holds the language-independent facts; `oc_product_description` holds the words, keyed on `(product_id, language_id)`. The split makes a price that differs by language impossible to express.

- The admin form's tabs are very nearly the table list. Create one product and find it in three tables; every later chapter assumes you can.
- A parent pointer cannot answer "everything beneath this category" in one query, so `oc_category_path` stores the answer — a deliberate denormalisation buying read speed with write complexity.
- The category listing query does `SELECT DISTINCT *` with four correlated subqueries, sorts on an unindexed column, and the whole join runs twice because the page needs a count.
- `ORDER BY LCASE(name)` cannot use the index on `name`. A function around a column hides the column from the optimiser.
- Measured: **one category page runs 146 queries**, and that number barely moves between a hundred products and fifty thousand. The $N + 1$ is per displayed product, not per catalogue product.
- Five hundred times the catalogue cost about 2.5× the page time, because `LIMIT 20` bounds the work.
- The three missing indexes cut database time by 45–62% at every size, and the page time did not reliably follow — the gain is real and smaller than the run-to-run noise. Measure the database to see it.
- An index makes one query faster; it cannot make 145 others disappear. The page's real problem is structural, and Chapter 20 meets it with caching.

Review Questions

1. **(MCQ)** The primary key of `oc_product_description` is: (a) `product_id`; (b) `(product_id, language_id)`; (c) `name`; (d) an auto-increment id.

2. **(MCQ)** `Using filesort` in `EXPLAIN` means: (a) the query wrote a temporary file to disk; (b) the result had to be sorted after fetching, because no index gave that order; (c) the table has no primary key; (d) the query failed.

3. **(Short)** Give the four questions that decide which table a product fact belongs in, with one example each.

4. **(Short)** Explain why `WHERE YEAR(date_added) = 2026` cannot use an index on `date_added`, and rewrite it so it can.

5. **(Short)** What is a materialised path, and what has to happen when a category is moved in the admin panel?

6. **(Applied)** A shop wants products to carry a "minimum order quantity" that differs per customer group. Say which existing table this belongs in or what new table is needed, give the columns and the primary key, and justify it against the four questions.

7. **(Applied)** The measurement in this chapter found that removing a full table scan and a filesort did not speed the page up. Design an experiment that would locate where the time actually goes, using only what is in your container. State what you would vary, what you would hold fixed, what you would record, and what result would point at each of the four layers in Figure 5.1.